



MongoDB

A document-oriented database

Prof. Alessandra Lumini

Alma Mater Studiorum - Università di Bologna



Getting started

Introduzione

- ❑ I database document-oriented sostituiscono il concetto di *riga* con un modello più flessibile: il **documento**
 - Possibilità di rappresentare relazioni **gerarchiche complesse in un unico documento**
 - Non esiste uno schema predefinito
- ❑ Alcune delle caratteristiche principali:
 - **Tanti indici a disposizione: composti, geo-spaziali, full-text**
 - Un meccanismo di **aggregation pipeline** per costruire aggregazioni complesse attraverso la concatenazione di piccoli «pezzi»
 - **Diversi tipi di collezione: time-to-live, fixed-size**
 - Possibilità di usare script nel linguaggio Javascript per manipolare i dati

Introduzione

❑ Alcuni concetti di base:

- Il **documento** è un insieme di «key-values», è più espressivo della sua controparte relazionale (i.e., la *riga*)
- La **collezione** è la controparte del concetto di *tabella*; diversamente da quest'ultima, non prevede uno schema di base
- Un'istanza di MongoDB può contenere diversi **database**, ognuno con le sue collezioni
- In ogni documento viene automaticamente inserito un campo speciale, identificato dalla chiave **_id**, il cui valore è unico all'interno della collezione (corrisponde alla chiave primaria)

MongoDB	Modello Relazionale
<i>Collezione</i>	Tabella
<i>Documento</i>	Riga
<i>Campo</i>	Colonna di una riga

Documenti

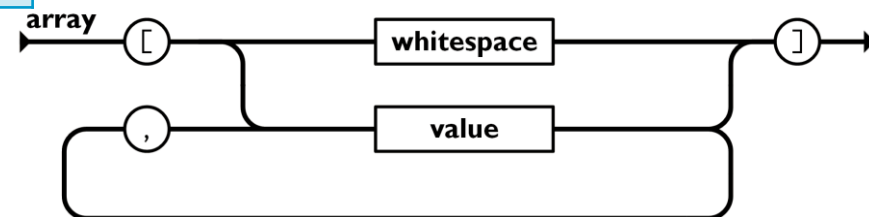
- ❑ I documenti corrispondono sostanzialmente a oggetti JSON, ma più espressivi
 - E' possibile utilizzare tipi di dato che il formalismo JSON non prevede
- ❑ In generale, sono **ricorsivamente** definiti **come oggetti composti da coppie ordinate chiave-valore**, in cui:
 - La *chiave* è una stringa case-sensitive
 - Non si possono usare i caratteri “.” e “\$”
 - Non possono esistere due chiavi identiche all'interno dello stesso oggetto
 - Il *valore* può essere di diversi tipi:
 - Un *tipo semplice* (e.g., stringa, numero, data, ecc.)
 - Un *altro oggetto*
 - Un *array* di valori
 - Generalmente, l'ordine delle chiavi non è importante

JSON: sintassi

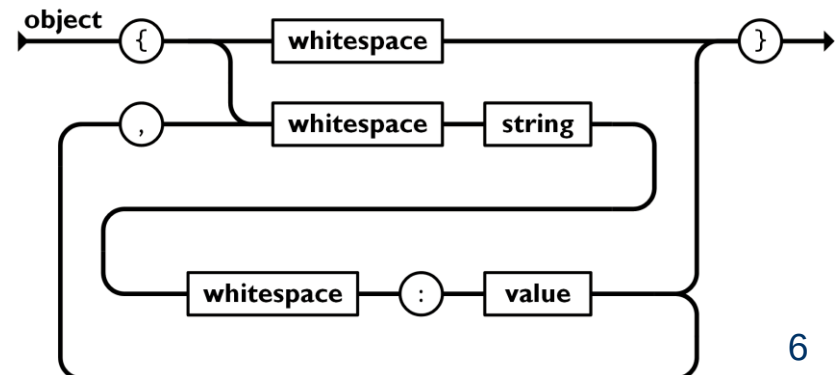
Stringa	"apple" "\u00c4pfel\n" " "
Numero	3 1.4 -1.5e3
Booleano	true false
Nullo	null
Array	["home", "wooden", 1, 3.14]
Oggetti	{ "id": 1, "name": "A wooden door", "tags": ["home", "wooden"] }

Un **valore** JSON può essere uno di:
String, Number, Boolean, Null,
Object, Array.

Un **array** è una raccolta ordinata di valori.



Un **oggetto** è una serie non ordinata di nomi/valori.



Documento

Field Values

- ❑ Scalar (Int, Boolean, String, Date, ...)
- ❑ Document (Embedding or Nesting)
- ❑ Array of JSON objects

Field Name (Key)

Field Value

```
{
  "_id": ObjectId("5037ee4a1084eb3ffeed7228"),
  "info": {
    "nome": "Enrico",
    "dataNascita" : ISODate("1988-08-04T20:42:00.000Z")
  },
  "interessi": ["Calcetto", "Viaggi", "Serie TV"],
  "didattica": [{
    "corso": "Big Data",
    "datore": "Università di Bologna"
  }, {
    "corso": "Introduzione ai sistemi di basi di dati",
    "datore": "FITSTIC"
  }]
}
```

Sub document {}

List []

Collezioni

- ❑ Una collezione è costituita da un insieme di documenti
- ❑ Non esiste uno schema di base
- ❑ Allora perchè creare più collezioni invece che tenerne una sola?
 - Comodità
 - Performance
 - Data locality
 - Indici diversi in collezioni diverse
- ❑ Una collezione è identificata da un nome
 - Non si può usare il carattere “\$”, ma si può usare il “.”, in particolare per organizzare concettualmente le collezioni in sotto-collezioni
 - E.g., blog.posts, blog.authors, ec..

MongoDB Model

Documento

```
{
  "_id": "59b99db4cfa9a34dcd7885b6",
  "name": "Ned Stark",
  "email": "ned@games.it",
  "groups": ["News", "Sport"]
}
```

Collezione

```
{
  " {
    " {
      "_id": "69c79db4cfa9a34dcd7885b6",
      "name": "Teo Black",
      "email": "teo@gmail.com",
      "groups": ["Computers", "Football"]
    }
  }
}
```

- Una Collezione è un insieme di documenti simili (non c'è il concetto di schema)
- All'interno di una collezione ogni documento ha un ID unico (**_id**, 12 byte) che viene generato automaticamente se non presente
- Non ci sono **vincoli di integrità referenziale**

Database

- ❑ Un'istanza di MongoDB può contenere tanti database, ciascuno dei quali può ospitare tante collezioni
- ❑ Ogni database ha il suo meccanismo di gestione dei permessi ed è salvato in un file dedicato
 - Di norma si utilizza un database per ogni applicazione
- ❑ I database sono identificati da un nome
 - Ci sono molte restrizioni sui caratteri (usare caratteri alfanumerici ASCII)

Database

- ❑ Alcuni database sono riservati

- ❑ **admin**

- E' il database principale in termini di **autenticazione**; gli utenti assegnati a questo database possono accedere anche a tutti gli altri
- Alcuni comandi possono essere eseguiti solo da questo database (e.g., elencare tutti i database, spegnere il server)

- ❑ **local**

- In un cluster, ne esiste **uno per ogni macchina** in cui è installato MongoDB
- E' usato per memorizzare dati che non devono essere distribuiti (es. startup_log)

- ❑ **config**

- Memorizza informazioni utili per l'utilizzo in modalità distribuita

Connessione via shell

- ❑ MongoDB mette a disposizione una **shell Javascript a linea di comando** con la quale interagire col database
- ❑ Come aprirla:
 - Aprire l'interfaccia a linea di comando di Windows (Start > `cmd`)
 - Andare nella cartella d'installazione della shell
 - Lanciare l'eseguibile
 - `Mongosh`
 - La shell si collega ad un database vuoto *test*
- ❑ La shell è sia un client MongoDB che un interprete Javascript

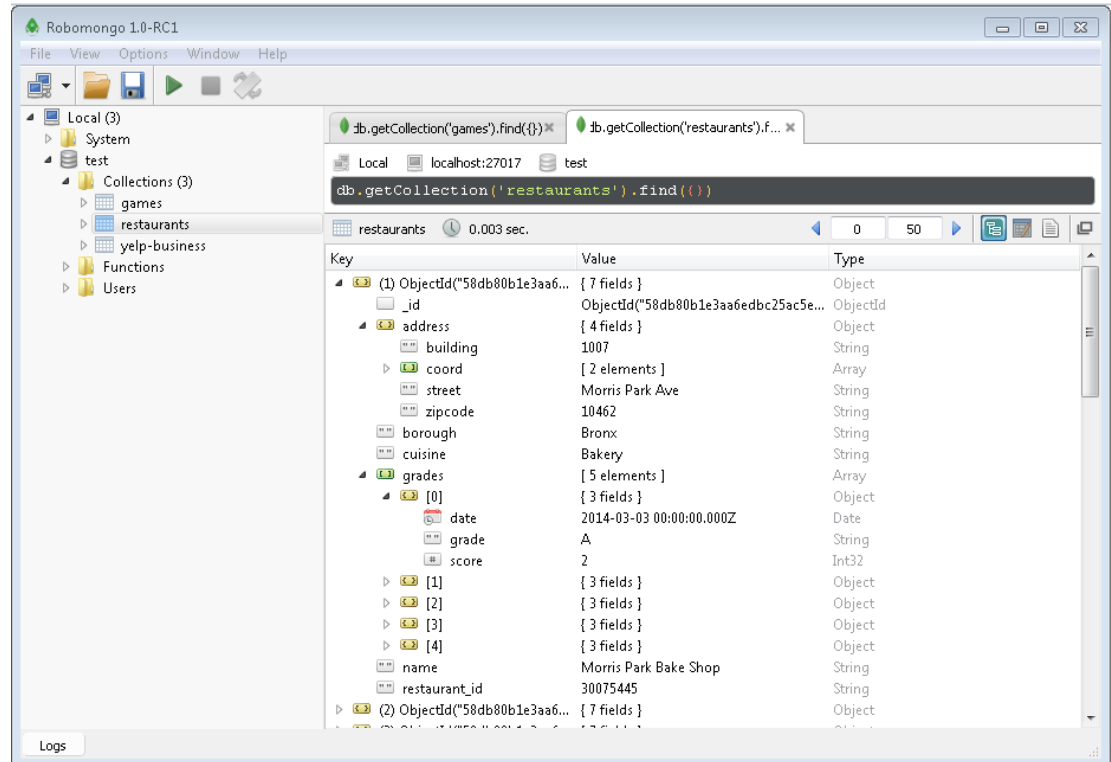
Robo3T

❑ Perché Robo3T?

- Semplifica la gestione e la navigazione del database
- Incorpora una shell di MongoDB

❑ Parametri:

- Connessione a localhost
- Porta 27017

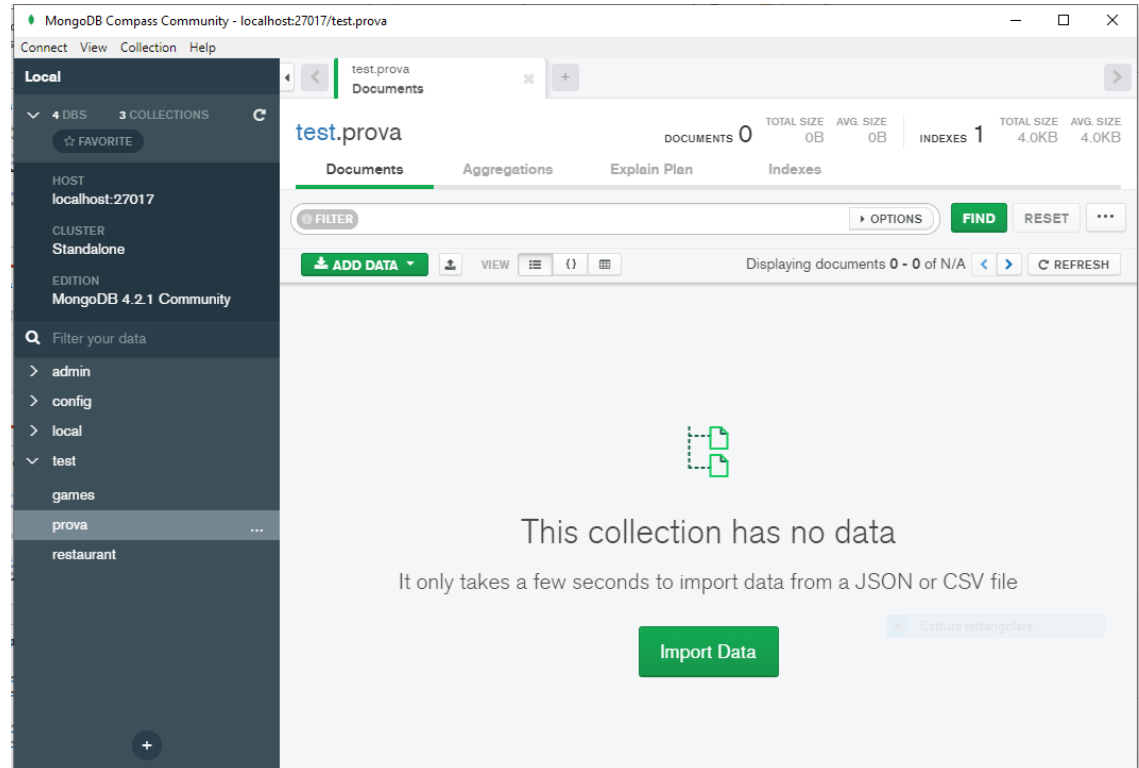


Robo3T è la nuova versione di Robomongo

MongoDB Compass

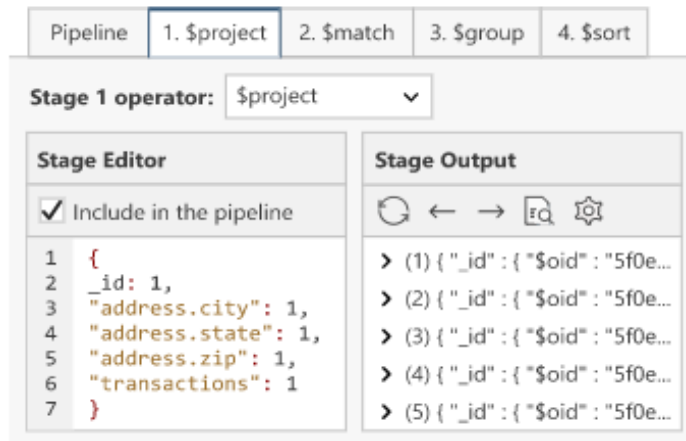
❑ Caratteristiche

- Semplifica insert e import
- Facilita query



- ## ❑ Parametri connessione a ATLAS (SRV string):
- `mongodb+srv://student:ADBstudent@alc-koahc.azure.mongodb.net/test`

Studio 3T



Pipeline 1. \$project 2. \$match 3. \$group 4. \$sort

Stage 1 operator: \$project

Stage Editor

☒ Include in the pipeline

```
1 {
2   _id: 1,
3   "address.city": 1,
4   "address.state": 1,
5   "address.zip": 1,
6   "transactions": 1
7 }
```

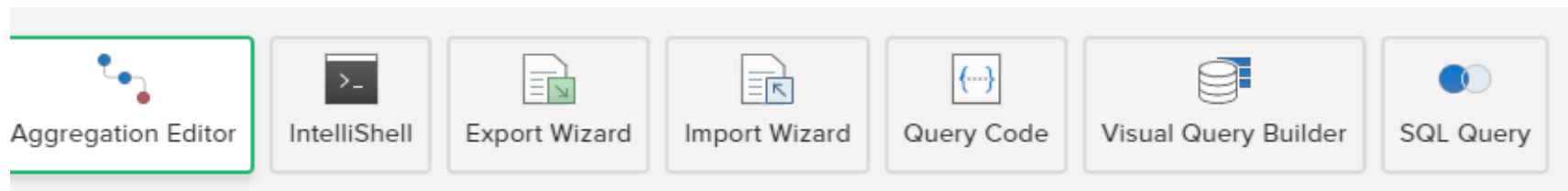
Stage Output

- > (1) { "_id" : { "\$oid" : "5f0e..."
- > (2) { "_id" : { "\$oid" : "5f0e..."
- > (3) { "_id" : { "\$oid" : "5f0e..."
- > (4) { "_id" : { "\$oid" : "5f0e..."
- > (5) { "_id" : { "\$oid" : "5f0e..."

Aggregation Editor

Not only can you add, edit, and move aggregation stages but also define operators and check inputs and outputs at each pipeline stage. This makes debugging easier and ensures that your query is accurate each step of the way.

[Learn more →](#)



Aggregation Editor

IntelliShell

Export Wizard

Import Wizard

Query Code

Visual Query Builder

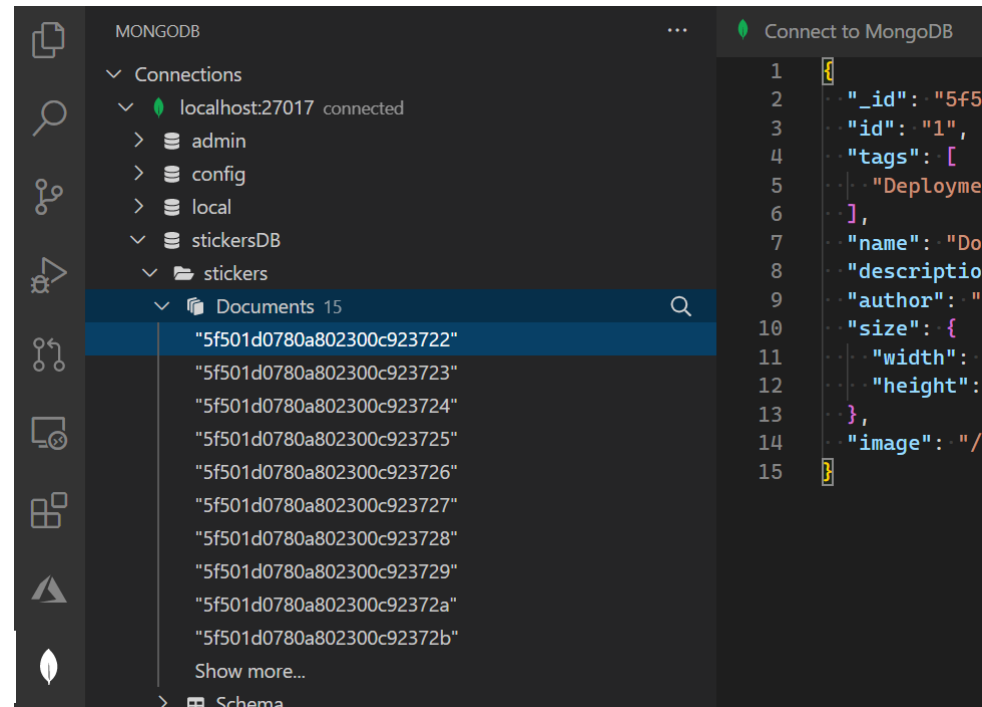
SQL Query

- ❑ Molto evoluto soprattutto per Aggregation Pipeline

MongoDB for VS Code

❑ Estensione a VS Code che consente:

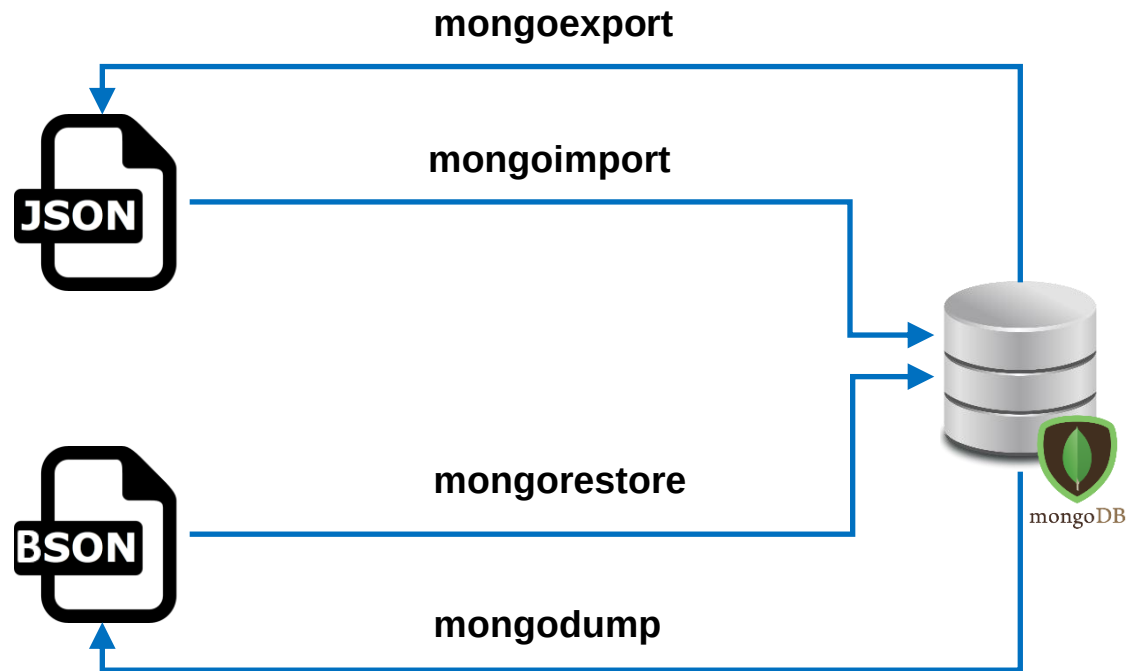
- Connessione a mongoDB
- Navigazione dati
- Playground
- Document Editing
- Accesso rapido alla shell



Connection String:

mongodb://localhost:27017/dba?readPreference=primary&ssl=false

Strumenti di import/export



❑ Eseguire da prompt

```
mongorestore --collection games --db dba games.bson
mongoimport --collection restaurants --db dba primer-dataset.json
mongoimport --collection nba2016players --db dba
nba2016_players.json
```

Comandi di base

- ❑ La maggior parte dei comandi di MongoDB sono metodi dell'oggetto `db`
- ❑ Di base, la shell si collega al database vuoto `test`
 - `db` – mostra il nome del database
 - `db. + tab` – mostra i metodi richiamabili
- ❑ Alcuni esempi
 - `db.getMongo().getDBs()` o `show dbs` – mostra i database presenti
 - `db.getCollectionNames()` o `show collections` – mostra le collezioni nel DB corrente
 - `use local` – passa al database *local*
 - `db.getSiblingDB("local")` – usa *local* senza modificare la variabile `db` nella shell
- ❑ Per lavorare su una collezione:
 - `db.[collectionName].[method]([parameters])`
- ❑ Help
 - `db.help()` / `db.[collectionName].help()` –Mostra tutte le funzioni che possono essere utilizzate con il database o la collezione



CRUD

Create, Read, Update, Delete

CRUD

❑ Create

- `db.collection.insert( <document> )` [insertOne,insertMany]
- `db.collection.update( <query>, <update>, { upsert: true } )`

❑ Read

- `db.collection.find( <query>, <projection> )`
- `db.collection.findOne( <query>, <projection> )`

❑ Update

- `db.collection.update( <query>, <update>, <options> )`

❑ Delete

- `db.collection.delete( <query> )`

CRUD (sintassi shell 5.0 +)

Operazione	Old shell	New shell
C reate	db.collection.insert()	insertOne(), insertMany()
R ead	db.collection.count()	countDocuments() estimatedDocumentCount()
U pdate	db.collection.update() db.collection.save()	updateOne(), updateMany() findOneAndUpdate()
D elete	db.collection.remove()	deleteOne() deleteMany() findOneAndDelete()

CRUD: Creazione Collezione

❑ Creare una nuova collezione

```
db.createCollection("users")
```

- Si può anche inserire subito il primo documento

```
db.users.insertOne ({  
    name: "mila",  
    age: 15,  
    status: "A"  
})
```

❑ Eliminare una collezione

```
db.users.drop()
```

SQL

```
CREATE users (  
    id INT NOT NULL,  
    name VARCHAR(30) ,  
    age NUMBER ,  
    status VARCHAR(5) ,  
    PRIMARY KEY(id)  
)
```

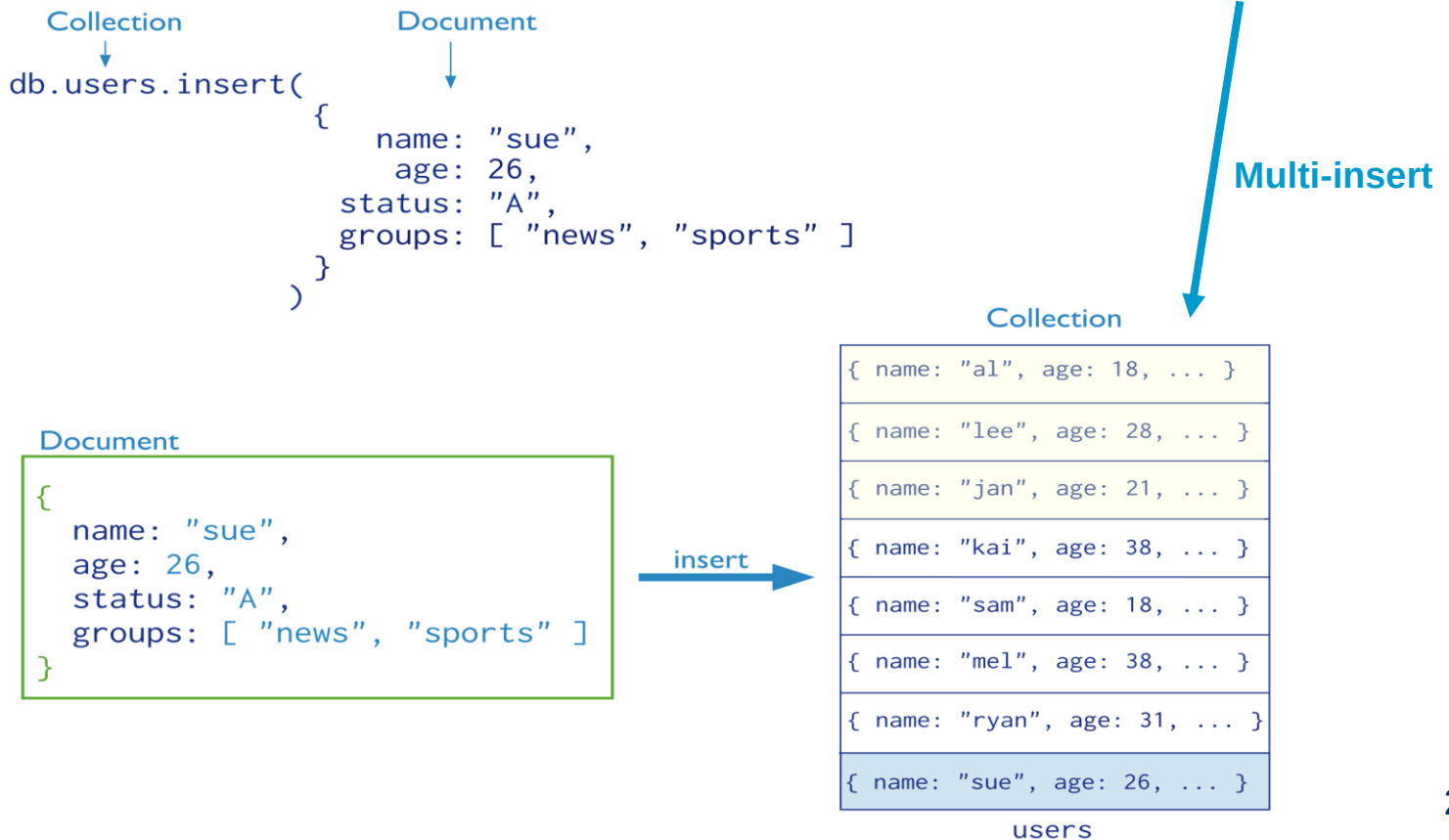
```
INSERT INTO users  
VALUES (1, "mila", 15, "A")
```

```
DROP TABLE users
```

CRUD: Inserimento

```
var myusers = [{name: "al", age: 18, status: "A"},  
{name: "lee", age: 28, status: "B"},  
{name: "jan", age: 21, status: "A"}];
```

```
db.users.insertMany(myusers);
```



Esercizio

❑ Creare una nuova collezione Studenti

1. Inserire alcuni studenti con il seguente schema

```
{nome: "Mario",  
cognome: "Rossi",  
età: "15",  
città: "Rimini",  
classe: "2B",  
voti: [9, 10, 9.5]}
```


CRUD: Read - Comandi principali

❑ Interrogazione dei dati

- **Find, FindOne** – modalità semplici per effettuare letture con proiezioni e selezioni
- **Count, Distinct** – modalità semplici per effettuare aggregazioni di dati
- **Aggregate** – modalità avanzata per effettuare aggregazioni di dati attraverso la concatenazioni di operazioni più semplici (match, unfold, group, ecc.)

❑ Forma base:

- `db.collezione.find(SELETTORE, PROIEZIONE).MODIFICATORE`

CRUD: Find

❑ Selezionare tutti i documenti

➤ `db.users.find()`

❑ Con un criterio di ricerca

➤ `db.users.find ({status: "A"})`

➤ La condizione può essere relativa a tutti i campi del documento `_id` incluso

❑ Con selezione e proiezione

```
db.users.find (  
  {status: "A"}, //criterio  
  {name:1, age:1 } //proiezione  
)
```

➤ `_id` viene sempre incluso a meno che non sia escluso esplicitamente

```
{name:1, age:1, _id:0 }) //proiezione
```

SQL

```
SELECT * FROM users
```

```
SELECT * FROM users WHERE  
Status = "A"
```

```
SELECT _id, name, age FROM  
users WHERE Status = "A"
```

CRUD: Modificatori

❑ Ordinare i documenti

```
db.users.find().sort({age:1 })
```

➤ 1= crescente, -1= decrescente

❑ Contare i documenti

```
db.users.find().count()
```

❑ Limitare l'output

```
db.users.find (
  {status: "A"}, //criterio
  {name:1,age:1 } //proiezione
).limit(5) //modificatore
```

❑ Solo il primo:

```
db.users.findOne (
  {status: "A"} //criterio
)
```

SQL

```
SELECT * FROM users
ORDER BY age
```

```
SELECT count(*) FROM users
```

```
SELECT _id,name,age FROM
users WHERE Status = "A"
LIMIT 5
```

```
SELECT * FROM users WHERE
Status = "A"
LIMIT 1
```

Esercizi

- ❑ Interrogare la collezione Studenti
- 2. Visualizzare tutti gli studenti di una classe
- 3. Visualizzare solo nome e cognome degli studenti
- 4. Ordinare gli studenti per età
- 5. Limitare l'output a soli 3 studenti

Scrivere ciascuna query a partire dalla precedente

CRUD: Update

- ❑ L'aggiornamento di un documento avviene in maniera atomica
 - Due aggiornamenti concorrenti sullo stesso documento vengono comunque serializzati; **l'ultimo che arriva, vince**
- ❑ Esistono diversi modi diversi per aggiornare i documenti
 - Aggiornamento dei campi attraverso «**modifiers**» (**Update**)
 - Opzione **upsert**
 - Da shell esiste anche l'operazione di **ReplaceOne** del documento con un altro documento

CRUD: Update

❑ Istruzione di base

```
db.<collection>.updateOne(           //updateMany
{<field1>:<value1>},                 //criterio
{$set: {<field2>:<value2>}})         //azione
```

❑ Esempio

```
db.users.updateMany(
{ age: 18 },
{ $set: { status: "A" } }
)
```

➤ OUTPUT:

```
WriteResult({"nMatched":2, "nUpserted":0, "nModified":2 })
```

SQL

```
UPDATE users
SET Status = "A"
WHERE age=18
```

❑ Il modificatore **\$set** esegue la modifica

CRUD: Upsert

□ Upsert= insert or replace

```
db.users.updateOne(  
  { name: "tom" },  
  { $set: {name: "tom",  
    age: 24,  
    status: "C",  
    hobbies : ["volley", "bike"]},  
    $setOnInsert : {createdAt : new Date() }  
  },  
  { upsert:true }  
)
```

//solo update

//insert

Attenzione: se lo stesso nome compare in più documenti, si rischia di aggiornare il documento sbagliato. In questo caso, è buona norma utilizzare l'`_id` come criterio di ricerca

CRUD: Modificatori

- ❑ I **modificatori** sono chiavi speciali che **specificano il tipo di modifica da effettuare** sui documenti
- ❑ I principali modificatori sono:
 - \$set, \$unset – **Imposta il valore** di un campo
 - \$inc – **Incrementa (o decrementa)** il valore di un campo numerico
 - \$push, \$addToSet – **Aggiunge** un valore ad un array
 - \$pop, \$pull – **Rimuove** un valore da un array
- ❑ Vanno nella sezione «azione»

```
db.<collection>.updateOne(           //collezione
{<field1>:<value1>},                 //criterio
{
  $set: {<field2>:<value2>},
  $inc: {<field3>:<amount3>},
  $unset: {<field4>:""},
}, {multi:true}                      //azioni e modificatori
                                     //opzioni
```


CRUD: Esempi

- ❑ `$push` aggiunge valori in fondo ad un array
 - Se il campo non esiste, viene creato
 - Se il campo esiste ma non è un array, viene prima convertito in un array e poi il campo viene aggiunto
 - E' possibile aggiungere più elementi con un unico comando utilizzando anche l'operatore `$each`
- ❑ `$addToSet` inserisce solo se l'elemento non è già presente, `$push` inserisce comunque

```
db.users.updateOne(  
  { name: "lee" },  
  {  
    $set: { status: "B" },  
    $inc: { age: 1 },  
    $currentDate: { lastModified: true },  
    $addToSet: { hobbies : { $each : ["food", "skate"] } }  
    // $addToSet: { hobbies : "skate" }  
  }  
)
```

CRUD: Modificatori \$pop e \$pull

- ❑ **\$pop** permette di **estrarre il primo o l'ultimo valore** nell'array (come se fosse una coda o una pila)
- ❑ **\$pull** permette di **estrarre uno o più valori** in base a determinati criteri
- ❑ **Esempi:**
 - `db.lists.update({}, {$pop : {todo: 1} } )`
 - Estrae l'ultimo elemento nell'array todo
 - `db.lists.update({}, {$pop : {todo : -1} } )`
 - Estrae il primo elemento nell'array todo
 - `db.lists.update({}, {$pull : {todo : "laundry"} } )`
 - Estrae tutti gli elementi nell'array todo che hanno valore "laundry"

CRUD: Delete

❑ Rimuovere tutti i documenti

- `db.users.deleteMany()` ,
`db.users.deleteOne()`

SQL

```
DELETE FROM users WHERE  
Status = "A"
```

❑ Con un criterio

- `db.users.deleteOne({status: "A"})`
- La condizione può essere relativa a tutti i campi del documento `_id` incluso

❑ Cancellare una collezione

- `db.users.drop()`
- Svuotare una collection cancellandola e ricreandola richiede molto meno tempo rispetto alla cancellazione di tutti i documenti

❑ Attenzione: non esiste un rollback! Una cancellazione è per sempre...

Esercizi

❑ Modificare la collezione Studenti

6. Aggiungere il campo "scuola" e "lastModified" a tutti i documenti
7. Aumentare la classe da "2B" a "3B" e l'età degli studenti
8. Aggiungere un voto a un studente (dati nome e cognome)
9. Eliminare tutti gli studenti di una classe

Utenti

- ❑ Un utente in MongoDB è definito da:
 - username e password e da una serie di ruoli che definiscono i suoi privilegi di accesso, sia a livello di database che di collezione.
- ❑ Le differenze con DB relazionali:
 - **Privilegi granulari:** In MongoDB, è possibile concedere privilegi di accesso a varie operazioni specifiche come “db.collection.find()”, db.collection.insert()”, “db.collection.update()” ecc., a differenza di SQL che offre un solo privilegio di accesso alla tabella.
 - **Utenti a livello di cluster:** MongoDB consente di creare utenti a livello di cluster che hanno accesso ad all'interno dei vari nodi di un cluster.

Utenti

read
readWrite
dbAdmin
userAdmin

...

db.createRole()

❑ Aggiungere un nuovo utente a un DB

```
db.createUser( {  
  user: "gruppo1", pwd: "gruppo",  
  roles: [ { role: "read", db: "test" } ]  
})
```

❑ Cambiare la password

```
db.changeUserPassword("gruppo1", passwordPrompt())  
db.changeUserPassword("gruppo1", "nuova")
```

Solo da shell

❑ Rimuovere un utente

```
db.dropUser("gruppo1")
```

❑ Elencare tutti gli utenti

```
db.getUsers()
```



Interrogazioni semplici

Find, findone, count, distinct

Il comando Find

- ❑ È il comando che permette di eseguire interrogazioni (query) sul DB e restituisce un **CURSORE**
- ❑ La forma di base è:
 - `db.collezione.find(SELETTORE, PROIEZIONE).MODIFICATORE`
- ❑ Dove
 - **collezione** va sostituito col nome della collezione da interrogare; corrispettivo SQL : FROM (ma limitato ad un'unica collezione)
 - SELETTORE è un (eventuale) oggetto che contiene i criteri di ricerca; corrispettivo SQL : WHERE
 - PROIEZIONE è un (eventuale) oggetto che contiene i campi su cui fare proiezione; corrispettivo SQL : SELECT list
 - MODIFICATORE è un (eventuale) metodo che si applica al **CURSORE** risultato del FIND (ordinamento, conteggio...)

Find – selezione semplice

- `db.restaurants.find()` //Restituisce tutti i documenti
- `db.restaurants.findOne()` //Restituisce solo il primo documento
- `db.restaurants.find({cuisine: "Hamburgers"})` //Restituisce i documenti in cui l'attributo cuisine (se presente) è valorizzato con la stringa "Hamburgers"
- `db.restaurants.find({}, {cuisine: 1})` //Restituisce tutti i documenti, ma proiettando solamente l'attributo cuisine (oltre all'_id, che viene restituito di default)
- `db.restaurants.find({cuisine: "Hamburgers"}, {cuisine: 1})` //La combinazione di selezione e proiezione

Find – selezione complessa

- ❑ L'espressione di condizioni di selezione complesse avviene attraverso l'incapsulamento di nuovi oggetti che rispettino la sintassi degli oggetti Javascript:
 - `nomeChiave : { operatore : valore }`
 - `operatore` corrisponde ad un operatore di confronto (e.g., `"$gte"`, acronimo di "Greater Than or Equal to")
 - `valore` corrisponde ad una costante (un numero o una stringa), o un oggetto, composto da un'altra coppia `operatore : valore`
- ❑ Esempio
 - Trova i ristoranti con `restaurant_id > "40356483"`
 - ~~`db.restaurants.find({restaurant_id >= "40356483"});`~~ //non funziona
 - `db.restaurants.find({restaurant_id: {$gte : "40356483"}});`

Find – operatori di confronto

❑ Quali sono gli operatori

- `$gte`, `$gt` – corrispondono a $\geq$ e $>$
- `$lte`, `$lt` – corrispondono a $\leq$ e $<$
- `$ne` – corrisponde a $\neq$

❑ Esempi

- `db.restaurants.find({name: {$gt : "Regi", $lt : "Regl"}});`
- `db.restaurants.find({restaurant_id: {$gte : "40356483", $lte : "40357500"}});`
- `db.restaurants.find({cuisine: {$ne : "Hamburgers"}} )`

Find – condizioni multiple

❑ Quali sono gli operatori

- `$in`, `$nin` – equivalenti alle clausole IN e NOT IN di SQL
- `$or`, `$nor`, `$and`, `$not` – equivalenti ai rispettivi operatori logici
- `$in`, `$nin`, `$or`, `$nor`, `$and` hanno come operando una lista []

❑ Esempi

- `db.restaurants.find({cuisine: {$nin : ["Hamburgers","Chicken"]} })`
- `db.restaurants.find({cuisine: {$in : ["Chinese", "Jewish/Kosher"]} })`
- `db.restaurants.find({$or: [{cuisine: {$in : ["Chinese", "Irish"]}] , {borough: "Brooklyn" }])`
- `db.restaurants.find({$and:[{cuisine:"American"},{borough:"Bronx"}]})`
- **Meglio:** `db.restaurants.find({cuisine:"American", borough:"Bronx"})`

Find – esistenza e campi nulli

- ❑ Alcuni attributi possono avere valore **null**
- ❑ Il comando
 - `db.collezione.find({"campo" : null})`
 - restituisce **sia** i documenti in cui la chiave «campo» esiste ed è valorizzata a null, **sia** i documenti in cui la chiave non esiste.
- ❑ Per avere solo i documenti in cui la chiave esiste ed è null, bisogna verificare anche l'esistenza :
 - Operatore **\$exists**,
sintassi: { campo: { \$exists: <booleano> } }
 - `db.collezione.find({"campo" : {"$in" : [null], "$exists" : true}})`

Find – interrogare stringhe

- ❑ Per eseguire il confronto tra stringhe (come con LIKE in SQL) si possono usare:
 - **Regular expression** { <field>: /pattern/<options> }
 - Il carattere **^** specifica che la stringa cercata è all'inizio, **\$** per la fine
 - {name: /a/} **//like '%a%'**
 - {name: /^pa/} **//like 'pa%'**
 - {name: /ro\$/} **//like '%ro'**
 - {name: { \$in: [/^a/, /^b/] } } **//nomi che iniziano per a o per b**
 - db.restaurants.find({ borough: /^Bro/ }).count() **//ristoranti del Bronx o Brooklyn**
 - L'opzione **/i** specifica che la ricerca è case insensitive
 - L'opzione **/m** specifica che la ricerca di posizioni iniziali ^ o finali \$ si ripete su tutte le righe di una stringa multiriga
 - { name: /aaa/i] } } **//nomi che includono la stringa aaa (case insensitive)**
 - db.restaurants.find({ name: /ALE/i }) **//ristoranti con nome che contengono la stringa ALE (case insensitive)**
 - **\$Regex**: operatore esplicito per Regular expressions
 - { <campo>: { \$regex: /pattern/, \$options: '<opzioni>' } }
 - { <campo>: { \$regex: /pattern/'<opzioni>' } }
 - db.restaurants.find({ name: { \$regex: /ALE/i } }) **//come sopra**

Esercizi

❑ Usare la collezione restaurants

10. Mostrare tutti i ristoranti esclusi quelli che si trovano a Brooklyn o nel Bronx
11. Mostrare i campi restaurant_id, name, borough e cuisine di tutti i documenti nella collezione.
12. Mostrare restaurant_id, name, borough e cuisine (escludere _id) di un ristorante che si trova a Queens
13. Mostrare i primi 5 ristoranti che si trovano nel Bronx
14. Mostrare i prossimi 5 (escludendo i primi 5) ristoranti che si trovano nel Bronx
15. Mostrare restaurant_id, name, borough e cuisine per tutti i ristoranti il cui nome inizia con 'Wil'

Find – interrogare array

❑ Find senza altri operatori

➤ Esiste un elemento

- { <campo>: <val> }
- restituisce il documento se **esiste** un elemento dell'array <campo> uguale al <val> specificato

❑ Operatori per interrogare array

➤ Tutti gli elementi

- { <campo>: { \$all: [<val1> , <val2> ...] } }
- restituisce gli array che contengono **tutti** gli elementi specificati

➤ Almeno un elemento

- { <campo>: { \$elemMatch: { <query1>, <query2>, ... } } }
- Restituisce un documento se **almeno** uno degli elementi dell'array soddisfa **tutte** le condizioni indicate da <query1>, <query2>.

➤ Dimensione

- db.collezione.find({ <campo>: { \$size: <val> } })
- Seleziona i documenti in cui l'array <campo> ha <val> elementi.

Esempi: interrogare array

□ Contesto: collezione **scores** con 3 documenti:

- { **_id**: 1, tests: 3, results: [82, 95, 88] }
- { **_id**: 2, tests: 6, results: [82, 98, 99] }
- { **_id**: 3, tests: 4, results: [75, 95, 82] }

□ Comandi

- `db.scores.find({results : 95})` //match se l'array contiene 95 (restituisce: 1 e 3)
- `db.scores.find({results : {$all : [82, 95]}})` //match se l'array contiene sia 82 che 95 (restituisce: 1 e 3)
- `db.scores.find({results : {$in : [82, 95]}})` //match se l'array contiene 82 o 95 (restituisce: 1, 2 e 3)
- `db.scores.find({results : [95, 82, 88]})` //match se l'array corrisponde esattamente a quello indicato (restituisce: nulla)
- `db.scores.find({results : {$size : 3}})` // match se l'array contiene 3 elementi (restituisce: 1, 2 e 3)
- `db.scores.find ({"results.2" : { $exists:true }})` // se l'array almeno 3 elementi
- `db.scores.find({"results.2" : 88})` //match se l'array contiene 88 in posizione 2 0-based (restituisce: 1)
- `db.scores.find({ results: { $elemMatch: { $gte: 97, $lt: 100 } } })` //match se l'array almeno un elemento tra 97 e 100 (restituisce: 2)

N.B. Per usare la notazione .
sono necessarie le " "

Find – interrogare array

- ❑ Quando si inseriscono più criteri su un **attributo con valore semplice** (e.g., una stringa o un numero), i criteri sono valutati in **AND**
 - `db.scores.find({tests : {$gt:5, $lt:7}})`
 - Il valore di `tests` deve essere maggiore di 5 e minore di 7
- ❑ Se l'**attributo è un array**, i criteri sono valutati per ogni elemento: se almeno uno elemento soddisfa ciascun criterio, il documento viene restituito
 - `db.scores.find({results : {$gt:93, $lt:96}})`
 - Almeno un elemento di `results` deve essere >93 e un (altro) elemento deve essere <96
- ❑ Per imporre i due vincoli in **AND** sul singolo elemento di un array, bisogna utilizzare l'operatore **\$elemMatch**
 - `db.scores.find({results : {$elemMatch: {$gt:93, $lt:96}}})`
 - Almeno un elemento di `results` deve essere >93 e <96

Find – interrogare array

- ❑ In fase di **proiezione** è possibile limitare il numero di elementi dell'array che vengono restituiti dalla query
- ❑ Contesto: un doc che contiene il post di un blog ed i relativi commenti
 - `db.blog.find(<criteria>, {comments: {$slice: 10}})`
 - restituisce i primi 10 commenti
 - `db.blog.find(<criteria>, {comments: {$slice: -10}})`
 - restituisce gli ultimi 10 commenti
 - `db.blog.find(<criteria>, {comments: {$slice: [23,10]}})`
 - salta i primi 23 documenti e restituisce i 10 successivi (dal 24° al 33°)
- ❑ Attenzione: **\$slice** da solo **NON** fa una proiezione su "comments" e **NON** filtra
- ❑ Per filtrare solo i commenti che soddisfano le condizioni:
 - `db.blog.find(<criteria>, {"comments.$": 1})`
 - restituisce SOLO i commenti che rispondono ai criteri di selezione indicati
 - Proiezione del risultato su «comments»

Find – interrogare oggetti

□ Contesto

- `{writer: {first:"Joe", middle:"K" , last:"Shoe" }}`

□ Esistono due modalità

- `db.people.find({writer : {first:"Joe", last:"Shoe"}})`
- Match esatto: l'oggetto cercato deve essere **uguale** a quello specificato (in questo caso, non restituisce nulla)
- `db.people.find({"writer.first":"Joe", "writer.last":"Shoe"})`
- In alternativa, si può usare la dot notation per referenziare i singoli campi (in questo caso, restituisce il documento)

Find – oggetti in un array

□ Contesto

```
{content : "...",  
  comments : [{author : "joe", score: 3, comment : "nice"},  
               {author : "mary", score: 6, comment : "terrible" }]}
```

□ Obiettivo: cercare i commenti di Joe con un punteggio ≥ 5

- `db.blog.find({comments : {author : "joe", "score" : {$gte : 5}}})`
- Sbagliato: cerca il match esatto (non va bene per un array)
- `db.blog.find({"comments.author" : "joe", "comments.score" : {$gte : 5}})`
- Sbagliato: le condizioni sono valutate sui diversi commenti della lista (esiste almeno un commento che soddisfa ciascuna condizione):
- `db.blog.find({comments : {$elemMatch : {author : "joe", score : {$gte : 5}}}})`
- Corretto: deve esistere un commento che soddisfa entrambe le condizioni

Find: modificatori

- ❑ **Limit**: restituisce solo i primi n documenti
 - `db.users.find().limit(3)`
- ❑ **Skip**: salta i primi n documenti e restituisci i successivi
 - `db.users.find().skip(3)`
- ❑ **Sort**: ordina i risultati sulla base di uno o più attributi
 - `db.users.find().sort({ age : -1, status : 1 })`
- ❑ Questi comandi possono essere combinati (ad esempio per gestire la paginazione dei risultati)
 - `db.users.find().limit(20).sort({name:1})`
 - `db.users.find().limit(20).skip(20).sort({name:1})`
 - ...
- ❑ Il comando **sort** viene sempre eseguito per primo

Count e Distinct

- ❑ **Count** conta i documenti restituiti da una query
 - `db.users.count({status : "A"})`
 - `db.users.find({status : "A"}).count()`
- ❑ **Distinct** restituisce i valori distinti di un campo
 - `db.collection.distinct(<field>, <query>)`
 - **Field** è il campo di proiezione, **query** di selezione
 - `db.users.distinct( "status", {hobbies : {"$exists" : true} } )`
 - Restituisce i valori distinti del campo "status" per gli utenti in cui è presente il campo "hobbies"
 - Se "status" è un array, vengono restituiti i valori distinti anche rispetto all'array di un singolo documento

Cursori

- ❑ Tutte le query restituiscono un cursore
 - Se non viene assegnato a una variabile, viene stampato
 - `var myCursor=db.users.find();`
- ❑ Metodi per scorrere i cursori (e visualizzarli)
 - `myCursor //visualizza il cursore`
 - `myCursor.forEach(printjson);`
 - `myCursor.forEach(function(o){print(o.name );} );`
 - `while (myCursor.hasNext()) {
 printjson(myCursor.next()); };`
- ❑ Assegnare un cursore a un array
 - `var docArray = myCursor.toArray();`
 - `var myDoc = docArray[3];`
- ❑ Altri metodi per i cursori:
 - <http://docs.mongodb.org/manual/reference/method/js-cursor/>

Esercizi

❑ Usare la collezione restaurants

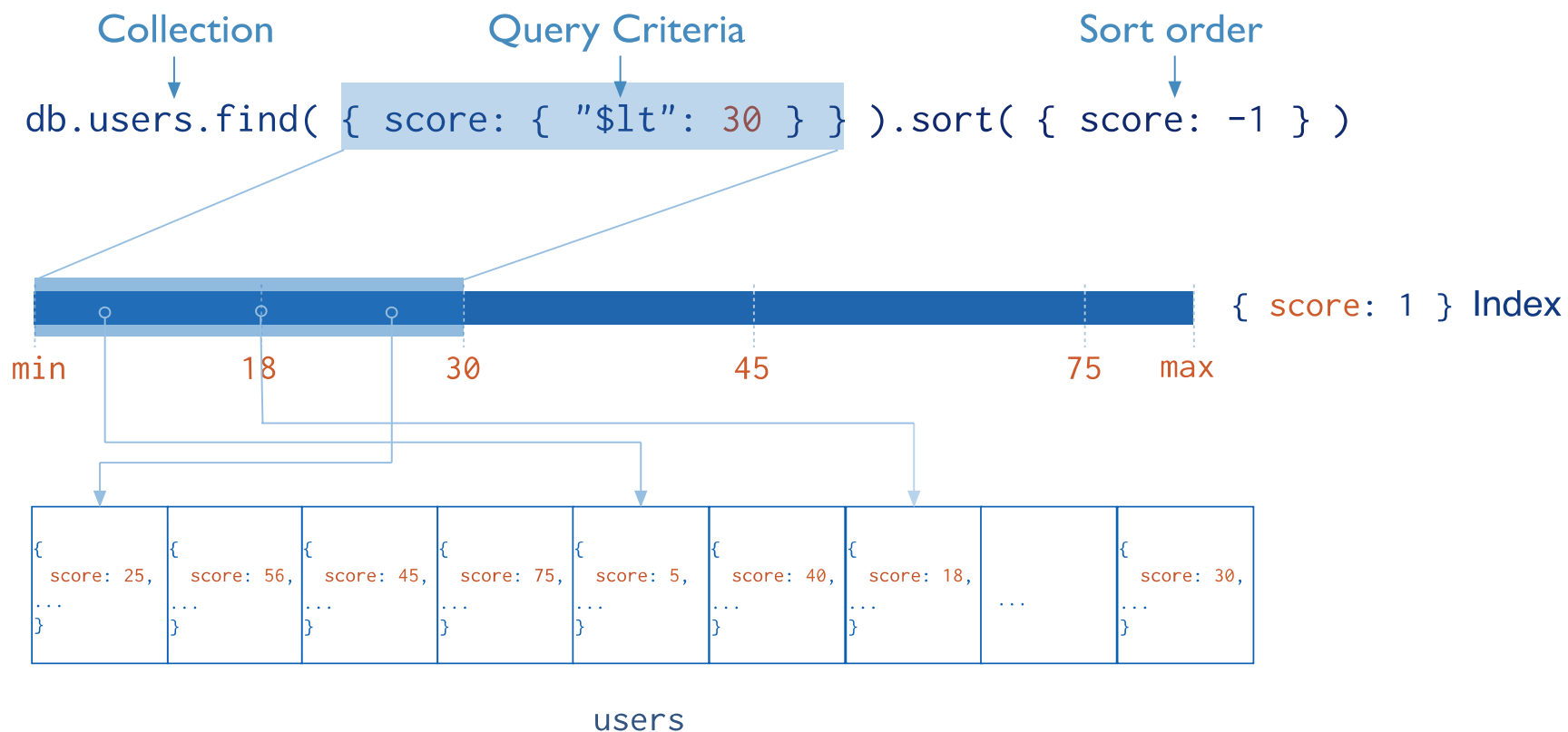
16. Mostrare i ristoranti che hanno almeno un score >90
 - 16bis: proiezione sui dati utili (nome, solo grades con score >90)
 - 16ter. Mostrare i ristoranti che hanno un score >90 e <100
17. Mostrare i ristoranti che hanno cucina diversa da "American", score >70 e latitudine (address.coord.0) <-65
18. Mostrare i ristoranti che non hanno avuto un score >10
19. Contare i ristoranti nel cui indirizzo non è specificata la street
20. Contare i ristoranti che hanno 6 voti (grades)
21. Contare i ristoranti che hanno più di 6 voti
22. Visualizzare tutti i tipi distinti di cucina



Indici

Indici

- ❑ MongoDB supporta la costruzione di indici a livello di collezione (su qualsiasi campo o sottocampo)
- ❑ **Sintassi:** `db.collection.createIndex(<campo>,<opzioni>)`
 - `db.users.createIndex(score:1)`



Indici

- ❑ La costruzione di indici consente di migliorare le performance in lettura
 - **ATTENZIONE:** gli indici vengono aggiornati ad ogni modifica; il rischio è di peggiorare le performance in scrittura
- ❑ **Indice composto:** `db.products.createIndex({cat: 1, price: -1})`
 - `:1` indica un ordinamento crescente, `-1` decrescente
 - L'ordine dei campi è importante
- ❑ **Caratteristiche**
 - Si possono indicizzare gli array
 - Esistono tipi speciali di indici: **geospaziali**, di **testo**, **hash**
 - Indici **parziali**: indicizzano solo alcuni documenti sulla base di una query
 - Indici **TTL**: cancellano il documento se la data indicizzata è obsoleta

Indici di testo

- ❑ Consentono di fare ricerche di testo all'interno di uno o **più** campi
 - Si può creare un solo indice di testo per collezione (ma su più campi)
 - `db.stores.createIndex({name:"text",description:"text"})` // elenca i campi da indicizzare
 - `db.stores.createIndex({ "$**": "text" })` // tutti i campi di testo
- ❑ L'indice esclude *stop-words* e riduce i termini a *radice*
 - Es.: "Collezione con indice di testo." → ["collezione", "indic", "test"]
- ❑ Caratteristiche
 - Si possono indicizzare campi testuali e array di stringhe
 - Si possono assegnare pesi diversi a campi diversi
 - La lingua di default è l'inglese, ma si può cambiare
 - `db.blog.createIndex({content:"text"},{default_language: "italian"})`

Operatore \$text

- ❑ `db.stores.insertMany([
 { _id: 1, name: "Java Hut", description: "Coffee and cakes" },
 { _id: 2, name: "Burger Buns", description: "Gourmet hamburgers" },
 { _id: 3, name: "Coffee Shop", description: "Just coffee" },
 { _id: 4, name: "Clothes Clothes", description: "Discount clothing" },
 { _id: 5, name: "Java Shopping", description: "Indonesian goods" }])`
- ❑ `db.stores.createIndex( { name: "text", description: "text" } )`

Operatore \$text

- ❑ `db.stores.find( { $text: { $search: "java coffee shop" } } )`
 - Le parole indicate sono considerate in OR
- ❑ `{ _id: 1, name: "Java Hut", description: "Coffee and cakes" },`
`{ _id: 2, name: "Burger Buns", description: "Gourmet hamburgers" },`
`{ _id: 3, name: "Coffee Shop", description: "Just coffee" },`
`{ _id: 4, name: "Clothes Clothes", description: "Discount clothing" },`
`{ _id: 5, name: "Java Shopping", description: "Indonesian goods" }`

Operatore \$text

- ❑ `db.stores.find( { $text: { $search: "java \"coffee shop\"" } } )`
 - Le stringhe indicate tra virgolette (") vengono cercate esattamente
- ❑ ~~`{ _id: 1, name: "Java Hut", description: "Coffee and cakes" },`
`{ _id: 2, name: "Burger Buns", description: "Gourmet hamburgers" },`
`{ _id: 3, name: "Coffee Shop", description: "Just coffee" },`
`{ _id: 4, name: "Clothes Clothes", description: "Discount clothing" },`
`{ _id: 5, name: "Java Shopping", description: "Indonesian goods" }`~~
- ❑ Se la ricerca contiene una frase + altre parole viene cercata solo la frase

Operatore \$text

- ❑ `db.stores.find( { $text: { $search: "java shop -coffee" } } )`
 - Le parole precedute dal meno (-) causano l'esclusione del documento
- ❑ `{ _id: 1, name: "Java Hut", description: "Coffee and cakes" },`
`{ _id: 2, name: "Burger Buns", description: "Gourmet hamburgers" },`
`{ _id: 3, name: "Coffee Shop", description: "Just coffee" },`
`{ _id: 4, name: "Clothes Clothes", description: "Discount clothing" },`
`{ _id: 5, name: "Java Shopping", description: "Indonesian goods" }`

Operatore \$text

- ❑ `db.stores.find(`
 - `{ $text: { $search: "java coffee shop" } },`
 - `{ score: { $meta: "textScore" } }``).sort({ score: { $meta: "textScore" } })`
 - E'possibile ordinare i risultati sulla base di un punteggio di similarità con la query di ricerca

- ❑ `{ _id: 1, name: "Java Hut", description: "Coffee and cakes" },`
`{ _id: 2, name: "Burger Buns", description: "Gourmet hamburgers" },`
`{ _id: 3, name: "Coffee Shop", description: "Just coffee" },` ← 1°
`{ _id: 4, name: "Clothes Clothes", description: "Discount clothing" },`
`{ _id: 5, name: "Java Shopping", description: "Indonesian goods" }`



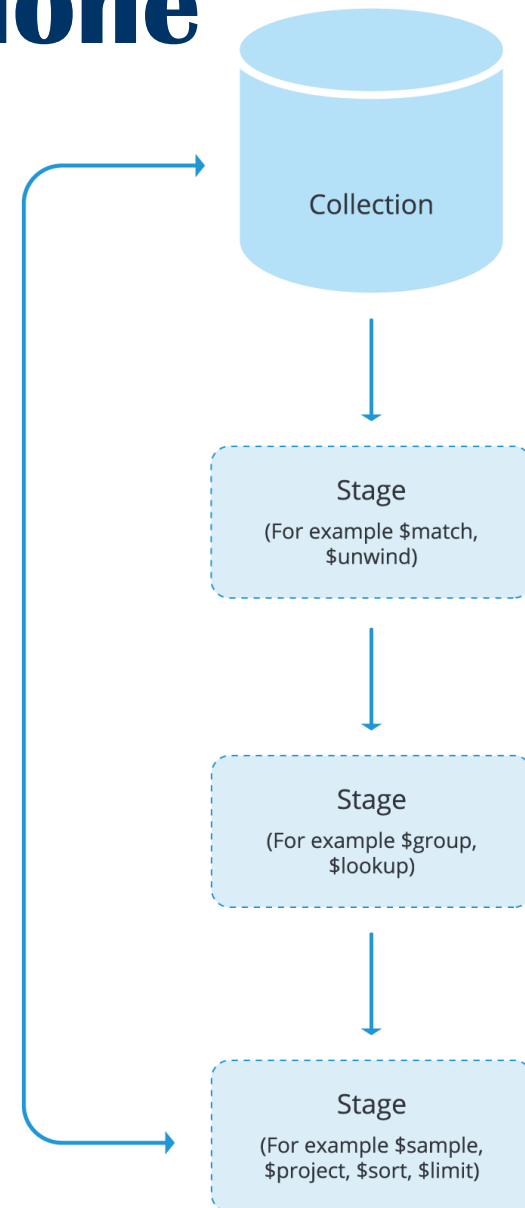
Framework di aggregazione

Aggregate

Framework di aggregazione

- ❑ Il framework di aggregazione permette di applicare **trasformazioni e aggregazioni** sui documenti di una collezione
- ❑ E' costituito da una serie di **operatori di pipeline**, *mattoni* che possono essere liberamente combinati tra loro (**anche più volte ed in qualunque ordine**) per dar vita ad interrogazioni più o meno complesse
 - Match, Project, Group, Unwind, Sort, Limit, Skip, Lookup
- ❑ **Non solo aggregazioni**: si possono esprimere interrogazioni che non si potevano fare col Find
 - Applicare trasformazioni sulle date
 - Concatenare due o più campi
 - Confrontare i valori di due campi
 - Restituire un singolo elemento di un array invece dell'array intero

Pipeline



Operatori di aggregazione

- ✧ `$geonear` → ordina i documenti dal più lontano al più vicino rispetto ad una posizione data
- ✧ `$match` → seleziona solo alcuni documenti che soddisfano le condizioni fornite in input
- ✧ `$project` → seleziona i campi prescelti
- ✧ `$group` → raggruppa in base ad uno o più campi
- ✧ `$limit` → seleziona i primi n documenti
- ✧ `$sort` → ordina il risultato in base ad alcuni campi
- ✧ `$out` → scrive l'output su una collezione
- ✧ `$unwind` → «srotola» un array
- ✧ `$lookup` → effettua un join
- ✧ ...

Esempio pipeline

❑ Collezione Students

```
{ "name": "Gadaffy", "unit": "A", "age": 18, "marks": [20, 50, 38] },  
{ "name": "John", "unit": "B", "age": 17, "marks": [38, 60, 70] },  
{ "name": "David", "unit": "A", "age": 30 },  
{ "name": "Emily", "unit": "C", "age": 18, "marks": [40, 87, 34] },  
{ "name": "Cynthia", "unit": "B", "age": 16, "marks": [60, 90, 98] },  
{ "name": "Mary", "unit": "B", "age": 28, "marks": [52, 50, 56] }
```



All documents in the collection are passed

❑ Interrogazione:

Seleziona nome e età degli studenti di più di 20 anni e ordina per età

```
db.students.aggregate([  
  { $match: { age: { $gt: 20 } } },  
  { $project: { _id: 0, name: 1, age: 1 } },  
  { $sort: { age: 1 } }  
])
```

➤ Alternativa (find):

```
db.students.find( {age: { $gt: 26 } }, { _id: 0, name: 1, age: 1 }).sort({age: 1});
```

\$match stage returns

```
{ "_id": ObjectId("5b47c275106c21d21c776e86"), "name": "David", "age": 30 }  
{ "_id": ObjectId("5b47c275106c21d21c776e89"), "name": "Mary", "age": 28 }
```

Only documents with age greater than 20 are passed

\$project stage returns

```
{ "name": "David", "age": 30 }  
{ "name": "Mary", "age": 28 }
```

Documents are arranged in ascending order using age field

\$sort stage returns

```
{ "name": "Mary", "age": 28 }  
{ "name": "David", "age": 30 }
```

Operatore \$match

- ❑ L'operatore **\$match** permette di **filtrare i documenti**
 - Opera sostanzialmente come una query di Find
 - Unica eccezione: non supporta operatori geospaziali
- ❑ E' buona norma utilizzare l'operatore il prima possibile
 - Riduce il numero di documenti delle operazioni successive
 - Può sfruttare gli indici (in fasi successive potrebbero non essere utilizzabili)
- ❑ Un esempio
 - `db.restaurants.aggregate([{$match:{cuisine:"Irish"}}])`
 - `db.restaurants.find({cuisine: "Irish"})`

Operatore \$project

- ❑ L'operatore **\$project** permette di effettuare una proiezione dei campi
 - E' **molto più potente** della proiezione nel comando Find
 - Permette di estrarre campi da oggetti innestati e di applicare trasformazioni
- ❑ Esempi di proiezioni semplici
 - `db.students.aggregate([{$project:{ "name":1,"_id":0}}]);`
 - Restituisce il nome dello studente ed esclude il campo `_id`
 - `db.students.aggregate([{$project:{ "userId" : "$_id", "name" : 1, "_id" : 0}}]);`
 - Rinomina il campo `_id` in `userId`
 - In pratica, introduce un nuovo campo `userId` il cui valore corrisponde al valore di `_id`
 - l'utilizzo dell'operatore `$` in `"$_id"` permette di indicare il riferimento ad un campo; altrimenti, `"_id"` verrebbe interpretato come un semplice valore

Operatore \$project – espressioni matematiche

- ❑ **Espressioni su 1 o più valori:** \$add, \$multiply
 - **\$add** : [expr1, expr2, ..., exprN]
- ❑ **Espressioni su 2 valori:** \$subtract, \$divide, \$mod
 - **\$subtract** : [expr1, expr2]
 - **\$divide**: divide il primo valore per il secondo
 - **\$mod**: divide il primo valore per il secondo e restituisce il resto
- ❑ **Un esempio**
 - `db.employees.aggregate([ { $project : { "totalPay" : { $subtract : [ { $add : [ "$salary", "$bonus" ] }, "$tax" ] } } } ] )`
 - Restituisce un campo calcolato: $\text{totalPay} = (\text{salary} + \text{bonus}) - \text{tax}$

Operatore \$project – espressioni sulle date

- ❑ Ci sono diverse espressioni che permettono di estrarre una specifica informazione a partire da una data

- "\$year", "\$month", "\$week"
- "\$dayOfYear", "\$dayOfMonth", "\$dayOfWeek"
- "\$hour", "\$minute", "\$second"

- ❑ Esempi

```
db.bios.aggregate([ { $project : {  
  "name" :1, "YOB" : { "$year" : "$birth" } } } ] )
```

- Restituisce l'anno di nascita

```
db.bios.aggregate([ { $project : {  
  "name" :1, "age" : { "$subtract" : [ { "$year" : new Date() },  
    { "$year" : "$birth" } ] } } } ] )
```

- Restituisce l'età

- ❑ **Attenzione:** Un'operazione aritmetica tra due date restituisce un risultato in millisecondi

Operatore \$project – espressioni sulle stringhe

- ❑ "\$substr" : [*expr*, *startOffset*, *numToReturn*]
 - Restituisce una sottostringa della stringa passata come primo parametro; parte da *startOffset* e restituisce *numToReturn* byte
 - Attenzione alla codifica: un byte potrebbe non corrispondere ad un carattere
- ❑ "\$concat" : [*expr1*[, *expr2*, ..., *exprN*]]
 - Concatena le stringhe passate come parametri
- ❑ "\$toLower", "\$toUpper"
 - Restituiscono la stringa passata come parametro in tutte minuscole o maiuscolo.

❑ Esempio

```
db.bios.aggregate([  
  $project : {  
    "fullname" : {"$concat": ["$name.first", " ", "$name.last"]},  
    "e-mail" : {"$concat" : [{"$substr" : ["$name.first", 0, 1]},  
    ".", "$name.last", "@gmail.com"]}]])
```

- Restituisce il nome e cognome e un indirizzo email tipo a.lumini@gmail.com

Operatore \$project: array

- ❑ \$arrayElemAt: [<array>, <idx>]

- Restituisce un elemento di un array in posizione idx

- ❑ Esempio

```
db.bios.aggregate([  
  $project: {  
    name :1,  
    firstContrib: { $arrayElemAt: ["$contribs", 0 ] },  
    lastcontrib: { $arrayElemAt: ["$contribs", -1 ] }  
  }  
]);
```

- Restituisce il primo e l'ultimo contributo di ciascuna persona

Esercizi

❑ Usare la collezione games

- 23. Restituire (in campi separati) l'anno, il mese ed il giorno di ogni partita
- 24. Restituire un campo che indichi quanti anni fa è stata disputata ciascuna partita (sottrarre l'anno di new Date() dall'anno della partita)
- 25. Restituire l'anno, il mese ed il giorno di ogni partita e i nomi delle 2 squadre
- 26. Restituire, per ogni partita: i due team, il rispettivi punti e la differenza tra i punteggi delle due squadre
- 27. Data la query precedente, filtrare solo le partite in cui la differenza è di un unico punto e in cui ha vinto la squadra di casa (la squadra che ha vinto è il team0)

Operatore \$group

- ❑ L'operatore **\$group** permette di **raggruppare** i documenti sulla base di determinate chiavi e di **calcolare** dei valori aggregati. Alcuni esempi:
 - Contesto: misurazioni meteo minuto-per-minuto.
Query: umidità media per giorno
 - Contesto: collezione di studenti
Query: raggruppare gli studenti per unit
 - Contesto: collezione di utenti
Query: raggruppare gli utenti per città e stato
- ❑ I campi su cui si vuole raggruppare costituiscono le chiavi del gruppo

```
{"$group" : {"_id" : "$day"}}
{"$group" : {"_id" : "$unit"}}
{"$group" : {"_id" : {"state" : "$state", "city" : "$city"}}}
```

Operatore \$group ed operatori aritmetici

- ❑ Oltre a specificare le chiavi su cui raggruppare è possibile indicare una o più operazioni per calcolare valori aggregati.

- ❑ Gli operatori aritmetici sono due:

- "\$sum" : value
Produce la somma dei valori
- "\$avg" : value
Produce la media dei valori

- ❑ Un esempio completo

```
db.students.aggregate([  
  {$match: {age: {$gt:16}}},  
  {$group: {"_id": "$unit", "numStud" : {"$sum" : 1}} },  
  {$sort: {numStud: -1}}  
])
```

Equivale a un
COUNT

- Raggruppa gli studenti per «unit» e conta in numero di studenti per gruppo (ordinando per cardinalità dei gruppi)

Operatore \$group ed operatori su estremi

- ❑ Ci sono quattro operatori per ottenere gli "estremi" del dataset:

- "\$max" : *expr* ; "\$min" : *expr*

Esaminano tutti i documenti e restituiscono rispettivamente il massimo ed il minimo valore trovato

- "\$first" : *expr* ; "\$last" : *expr*

Esaminano solo il primo e l'ultimo documento per restituire il valore trovato

- ❑ Esempio

```
db.students.aggregate([{"$group" : {"_id" : "$unit",  
"lowestAge" : {"$min" : "$age"},    "highestAge" :  
{"$max" : "$age"}} }])
```

- Raggruppa gli studenti per unità e ne visualizza età massima e minima

Operatore \$group ed operatori di collezione

- ❑ Ci sono due operatori che consentono di costruire un array con i valori riscontrati in ciascun gruppo

- "\$addToSet" : *expr*

- Costruisce un array con tutti i valori distinti

- "\$push": *expr*

- Costruisce un array con tutti i valori trovati, anche duplicati

- ❑ Un esempio

```
db.students.aggregate([
  {$project : {"unit" : 1, "name" : 1}},
  {$group : {"_id":"$unit","numStud":{"$sum":1},
  "studentList" : { "$addToSet" : "$name"}}}  ]);
```

Restituisce l'elenco degli studenti in ciascun gruppo

Operatore \$unwind

- L'operatore **\$unwind** permette di *appiattare un array*, costruendo tanti documenti quanti sono gli elementi dell'array

```
{ $unwind: {  
    path: <field path>, //array da decomporre  
    includeArrayIndex: <string>, //opzionale  
    preserveNullAndEmptyArrays: <boolean> //opzionale  
}  
}
```

- E' utile per effettuare proiezioni e aggregazioni sugli elementi interni degli array

```
db.students.aggregate([  
    {$match: {"name": "John"}},  
    {$unwind: {path : "$marks",includeArrayIndex : "m_ix"}},  
    {$project: {"_id" : 0,"name" : 1,"mark" : "$marks", "m_ix":1}  
}])
```

➤ Restituisce i voti di John (con relativo ordine)

```
{"name" : "John", "mark" : 38.0, "m_ix" : 0}  
{"name" : "John", "mark" : 60.0, "m_ix" : 1}  
{"name" : "John", "mark" : 70.0, "m_ix" : 2}
```

Operatore \$unwind

❑ Un altro esempio:

```
db.students.aggregate([
  {$unwind: {path : "$marks"}},
  {$group :{"_id":"$unit","avgMark" : {"$avg" : "$marks"} } }])
```

➤ Restituisce la media voti per unit

```
{"_id" : "B", "avgMark" : 63.7}
```

```
{"_id" : "A", "avgMark" : 47.6}
```

```
{"_id" : "C", "avgMark" : 53.6}
```

➤ **Attenzione:** senza unwind il campo avgMark sarebbe null

❑ Se l'array contiene un altro array, è possibile applicare l'operatore \$unwind in cascata (prima sull'array esterno, poi su quello interno)

Operatori \$sort, \$limit e \$skip

- ❑ Gli operatori **\$sort**, **\$limit** e **\$skip** funzionano come nella formulazione delle interrogazioni semplici
 - Se si vuole ordinare un grande numero di documenti, è buona norma fare l'ordinamento il prima possibile lungo la pipeline e avere un indice sul campo
- ❑ E' possibile ordinare anche sui campi creati lungo la pipeline
- ❑ Un esempio

```
db.students.aggregate([
  {$unwind: {path : "$marks",includeArrayIndex : "m_ix"}},
  {$project: {"_id" : 0,"name" : 1,"mark" : "$marks",
    "m_ix":1}  },
  {$sort : {"mark" :-1}},
  {$limit : 5}])
```

 - Seleziona i 5 migliori voti

Operatore \$lookup

- ❑ Introdotto a partire dalla versione 3.2
- ❑ L'operatore **\$lookup** permette di eseguire il **left outer join** tra collezioni residenti nello **stesso database**
 - Nella collezione «primaria» viene creato un nuovo campo di tipo *array*, contenente gli eventuali documenti corrispondenti nella collezione «secondaria»
- ❑ La sintassi:

```
$lookup: {  
  from: "join_collection",  
  localField: "field from the input documents",  
  foreignField: "field from join_collection",  
  as: "output_array"}
```

Operatore \$lookup: esempio

❑ Collezioni **orders** e **inventory**

```
db.orders.insert([
  { "_id" : 1, "item" : "abc", "price" : 12, "quantity" : 2 },
  { "_id" : 2, "item" : "jkl", "price" : 20, "quantity" : 1 },
  { "_id" : 3 }])
db.inventory.insert([
  {"_id": 1, "sku" : "abc", description: "product 1", "instock" : 120 },
  {"_id": 2, "sku" : "def", description: "product 2", "instock" : 80 },
  {"_id": 3, "sku" : "ghi", description: "product 3", "instock" : 60 },
  {"_id": 4, "sku" : "jkl", description: "product 4", "instock" : 70 },
  {"_id": 5, "sku": null, description: "Incomplete" },
  {"_id": 6 }])
```



❑ Esempio: ricomporre ordini con i dati di inventario

```
db.orders.aggregate([
  { $lookup: {from: "inventory", localField: "item", foreignField:
    "sku", as: "inventory_docs" } }])
```

Operatore \$lookup: esempio

❑ Risultato

```
{ "_id" : 1.0, "item" : "abc", "price" : 12.0, "quantity" : 2.0,
  "inventory_docs" : [{ "_id" : 1.0, "sku" : "abc", "description" :
    "product 1", "instock" : 120.0 } ] }
{ "_id" : 2.0, "item" : "jkl", "price" : 20.0, "quantity" : 1.0,
  "inventory_docs" : [{ "_id" : 4.0, "sku" : "jkl", "description" :
    "product 4", "instock" : 70.0 } ] }
{ "_id" : 3.0,
  "inventory_docs" : [{ "_id" : 5.0, "sku" : null, "description" :
    "Incomplete" }, { "_id" : 6.0 } ] }
```

- ❑ NB: il terzo ordine fa match con entrambi i documenti di inventario per cui sku è null

Esercizi

❑ Usare la collezione games

28. Raggruppare per nome-squadra e calcolare la media punti fatta e subita (solo nelle partite vinte (team0))
29. Contare il numero di partite giocate per ogni mese, anno (ordinando il risultato su anno, mese)
30. Eseguire l'unwind dell'array teams e raggruppare per team per ottenere il totale di: partite giocate, vinte, perse, punti segnati
31. Eseguire l'unwind dell'array box e dell'array players, raggruppare per player per ottenere il totale di punti segnati; ordinare il risultato per visualizzare per primi i migliori giocatori del'NBA

Usare anche la collezione nba2016_players

32. Unire la collection games con la collection nba2016_players ed escludere i giocatori per cui non sono state trovati riferimenti (N.B.: se non ci sono riferimenti, il nuovo campo sarà un array vuoto)
33. Estendere la query precedente per visualizzare il nome del giocatore, il totale di punti fatti ed il valore suo contratto (contract.amount)

Viste in MongoDB

- ❑ Le viste in MongoDB sono collezioni a **sola lettura** simili a quelle dei DB relazionali
 - Sono il risultato di una query (find o aggregate)
 - Non sono persistenti
 - Sono create dinamicamente
 - Sono automaticamente modificate con la collezione base
 - Una volta create si interrogano come collezioni normali
- ❑ Sintassi (2 alternative):

```
db.createCollection(  
  "<viewName>",  
  {  
    "viewOn" : "<source>",  
    "pipeline" : [<pipeline>],  
    "collation" : { <collation> }  
  }  
)
```

```
db.createView(  
  "<viewName>",  
  "<source>",  
  [<pipeline>],  
  {  
    "collation" : { <collation> }  
  }  
)
```

Viste: esempio

- ❑ Esempio: vista che unisce ordini con inventario

```
db.createView(  
  "orders_view",  
  "orders",  
  [{  
    "$lookup" : {  
      "from" : "inventory",  
      "localField" : "item",  
      "foreignField" : "sku",  
      "as" : "inventory_docs" }},  
    {  
      "$project" : {  
        "_id" : 1.0,  
        "item" : 1.0,  
        "price" : 1.0,  
        "quantity" : 1.0,  
        "desc" : { "$arrayElemAt" : ["$inventory_docs", 0] } }},  
    {  
      "$project" : {  
        "_id" : 1.0,  
        "item" : 1.0,  
        "price" : 1.0,  
        "quantity" : 1.0,  
        "description" : "$desc.description",  
        "instock" : "$desc.instock" } }])
```

- ❑ Visualizza ordini che non possono essere evasi

```
db.orders_view.aggregate([{$match:{$expr:{$gt:["$instock", "$quantity"]}}}]])
```



Replication e Sharding

Replication e Sharding:

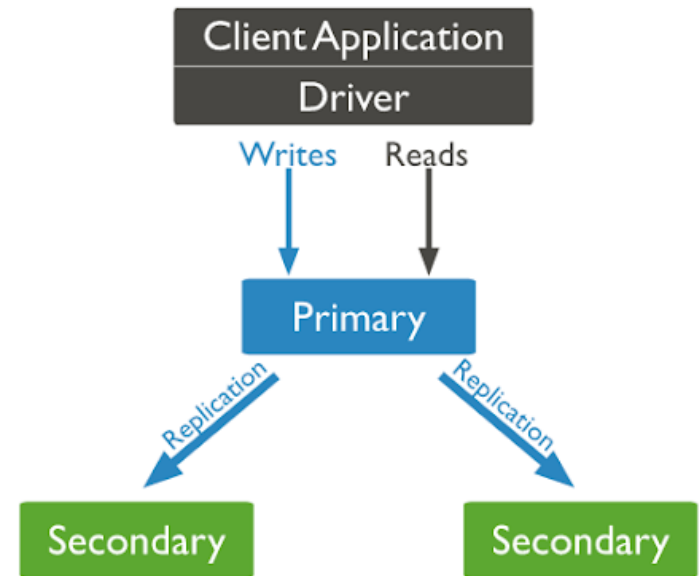
Scalabilità e High Availability

❑ Replication:

- duplicazione di dati per ridondanza e alta disponibilità.
- **Architettura Master-slave**

❑ Sharding:

- partizionamento di dati per scalabilità orizzontale.
 - Ranged Sharding
 - Hashed Sharding



Replication: assicurare la continuità del servizio

❑ Replica set:

- gruppo di nodi che contengono copie identiche del database.

❑ Nodo primario:

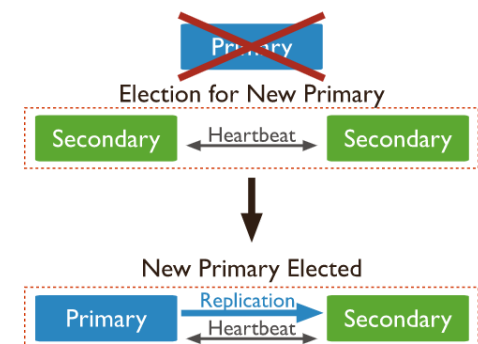
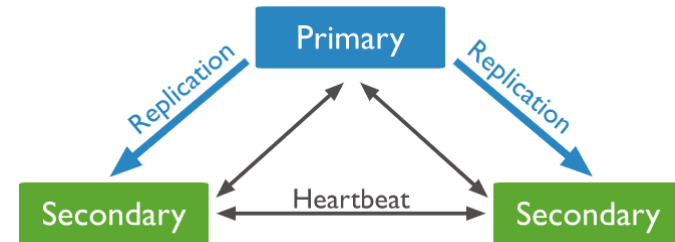
- riceve le scritture e le replica ai secondari.

❑ Nodi secondari:

- forniscono ridondanza e leggono i dati.

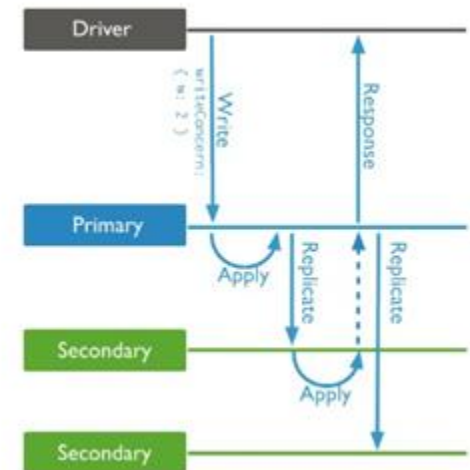
❑ Elezione del primario:

- automatico in caso di guasto del primario.
- **Elezione a maggioranza:** il nodo con il maggior numero di voti diventa il nuovo primario.
- **Elezione basata sul tempo:** il nodo con il timestamp più recente diventa il nuovo primario.



Replication: Write Concern

- ❑ La Write Concern in MongoDB definisce il livello di garanzia desiderato per le operazioni di scrittura
 - $w : 0$: Nessuna garanzia. La scrittura è completa non appena il server primario riceve la richiesta.
 - w : "majority" : La scrittura è completa solo quando è stata scritta correttamente sul server primario e sulla maggioranza dei secondari.
 - $w:N$: Maggioranza personalizzata.
 - $wtimeout$: L'operazione di scrittura attende la maggioranza per un tempo specificato prima di fallire.



```
db.collection.insertOne({name:"John"}, {writeConcern: {w:"majority"}});
```

Replication: vantaggi

- ❑ Tolleranza ai guasti:

- Se un nodo secondario fallisce, il database rimane disponibile.

- ❑ Alta disponibilità:

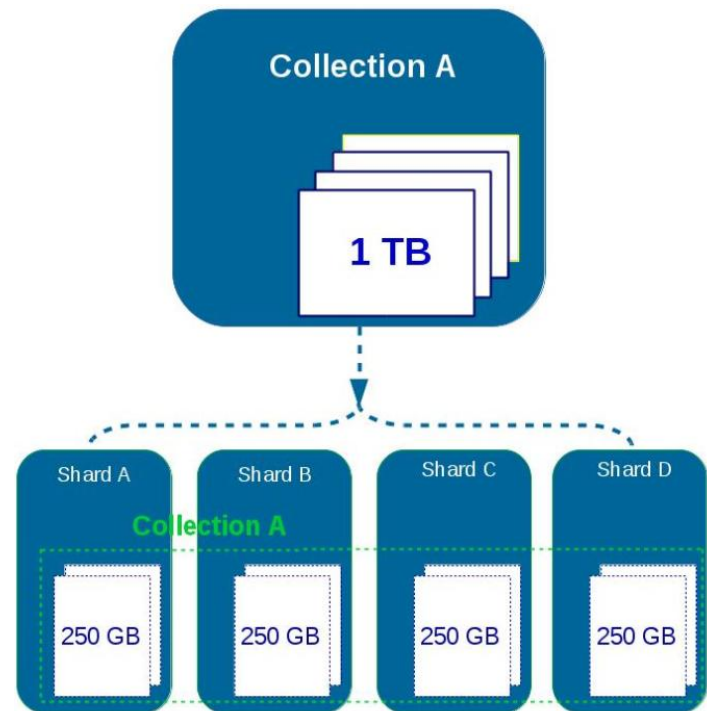
- Le letture possono essere servite da qualsiasi nodo del replica set.
- Le scritture possono essere reindirizzate al nodo primario se un nodo secondario non è disponibile.

- ❑ Ridondanza dei dati:

- I dati sono conservati su più nodi, proteggendoli da perdite accidentali.
- In caso di errore del disco o di corruzione dei dati su un nodo, i dati possono essere recuperati da un altro nodo.

Sharding

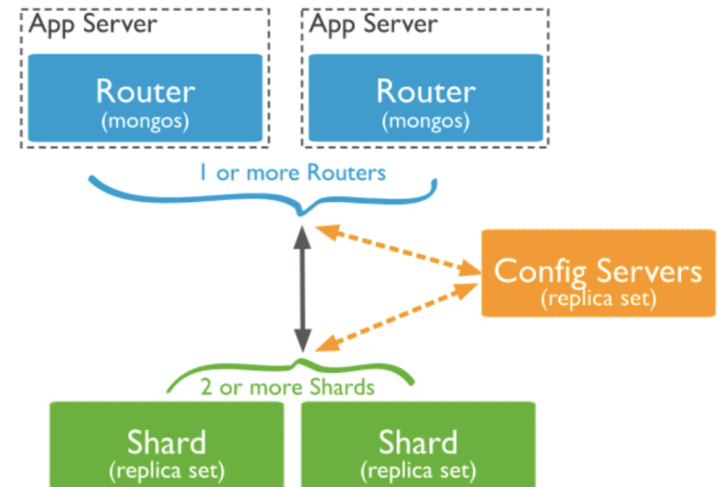
- ❑ Lo sharding (partizionamento orizzontale) è un metodo per distribuire i dati su più computer.
- ❑ MongoDB utilizza lo sharding per supportare distribuzioni con set di dati molto grandi e operazioni con throughput elevato.
- ❑ Vantaggi
 - Superare i limiti hardware
 - Migliorare la disponibilità
 - Migliorare le query
- ❑ Svantaggi
 - Complessità
 - Consistenza dei dati
 - Non adatto a tutte le query



Sharding: Architettura

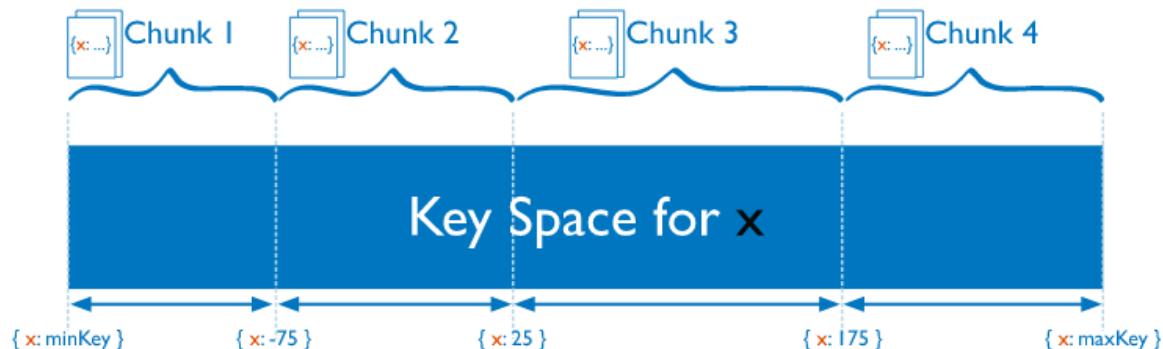
- Architettura:
- Shard (replica set):
 - Replica set che contiene una parte dei dati del cluster.
- Router (Mongos):
 - Funziona come un router di query per le applicazioni client, gestendo sia le operazioni di lettura che di scrittura.

I client si connettono a un router, non a singoli shard: il router invia le richieste agli shard pertinenti e aggrega il risultato in una risposta coerente per il client.
- Server di configurazione:



Sharding: Distribuzione e Equilibrio dei Dati

- **Shard Key:** la sua scelta è fondamentale per garantire una distribuzione uniforme dei dati e interrogazioni efficienti.
 - Meglio lo sharding per **range** per:
 - Frequenti query di range su shard key.
 - Dati ordinati sulla shard key.
 - Il carico è geograficamente distribuito
 - E' preferibile lo sharding per **hash** per:
 - Dati non ordinati sulla shard key
- **Chunk:** una porzione di dati all'interno di uno shard, definita dalla shard key. E' l'unità di migrazione dei dati tra shard durante ribilanciamenti.
- **Balancer:** Garantisce una distribuzione equilibrata dei dati tra gli shard.



Sharding: Benefici

➤ Maggiore capacità di lettura/scrittura:

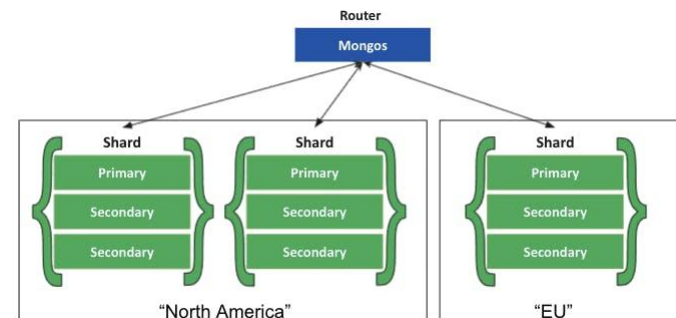
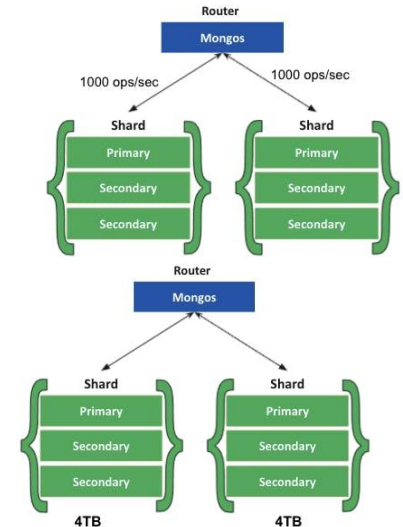
- Sfruttando il parallelismo la capacità di lettura/scrittura aumenta per ogni shard aggiuntivo.

➤ Maggiore spazio di archiviazione:

- Allo stesso modo aumenta la capacità di archiviazione complessiva. Se un singolo shard ha capacità di 4 TB, ogni shard aggiuntivo aumenterebbe la capacità totale di altri 4 TB.

➤ Località dei dati:

- Lo sharding per zone permette di creare database distribuiti per supportare applicazioni geograficamente diffuse, con policy che impongono la residenza dei dati all'interno di regioni specifiche. Ogni zona può avere uno o più shard.



Configurare un Replica Set

❑ Avviare i nodi del replica set

- Creare una directory dati per ciascun nodo
- Avviare 3 nodi MongoDB (su porte diverse)

```
Start mongod --port 27018 --dbpath /path/rs1 --replSet rsName
```

```
Start mongod --port 27019 --dbpath /path/rs2 --replSet rsName
```

```
Start mongod --port 27020 --dbpath /path/rs3 --replSet rsName
```

- avviare il client MongoDB shell (mongosh):

```
mongosh --port 27018
```

- Dentro il client shell inizializzare il replicaset:

```
rs.initiate({  
  _id: "rsName",  
  members: [  
    { _id: 0, host : "localhost:27018" },  
    { _id: 1, host : "localhost:27019" },  
    { _id: 2, host : "localhost:27020" }  
  ]})
```

Guida all'uso di un Replica Set

❑ Connessione (es. con VSCode)

```
mongodb://localhost:27018,localhost:27019,localhost:27020/mydata  
base?replicaSet=rsName
```

❑ Operazioni di Replica

➤ Monitoraggio dello stato del Replicaset

```
rs.status()
```

➤ Aggiunta di nuovi Membri:

```
rs.add("<nuovo_membro>:<porta>")
```

➤ Rimozione di Membri Esistenti:

```
rs.remove("<membro>:<porta>")
```

➤ Forzare una Elezione del Primary:

```
rs.stepDown()
```

```
rs.status()
```

Esercizio: Test di Replica Set

- ☐ Inserire alcuni dati nel database.
- ☐ Forzare la failover del nodo primario
- ☐ Verificare che il database sia ancora accessibile e che i dati siano stati replicati sul nuovo nodo primario.

Implementare lo Sharding

❑ Abilitare lo sharding

- Connettersi al database con un client mongo.
- Eseguire il comando:

```
db.adminCommand({ enableSharding: "databaseName" })
```

❑ Definire la shard key

- Scegliere un campo o un insieme di campi come shard key.
- Eseguire il comando:

```
db.adminCommand({ shardCollection: "collectionName", key: {  
  shardKeyField: 1 } })
```

❑ Distribuire i dati

- I dati verranno automaticamente distribuiti sugli shard in base alla shard key.
- È possibile monitorare la distribuzione dei dati con il comando `db.stats()`.